

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a17bdab31c3889b58.jsonl`
- SHA-256: `2dfd61924c127275b04cd0235c70dfb38ebefb53e77d6403698f0f0f98847f5`
- Source modified: `2026-06-13T19:16:25+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:15:31.813Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

```
FINDING: {"title":"Gate B (min_players person gate) in _select_for_handoff is completely
untested","severity":"high","file":"backend/pipeline/segmenters/fusion_hybrid.py:124 ;
tests/test_fusion_hybrid.py","detail": "_select_for_handoff has two gates. Gate A (min_crossings)
is covered by test_gate_a_drops_low_crossing_window_from_handoff_only. Gate B ? `if person_on
and min_players > 0 and st.get(\"players_in_court\", 0.0) < min_players: continue` ? has no test
at all. This includes the important guard that min_players is IGNORED when the person cue is OFF
(person_on=False). I verified by hand that the logic works (with person on + min_players=2 it
drops the 1-player window; with person off it keeps both), but the suite never asserts it, so a
regression that, e.g., applied min_players regardless of person_on, or read the wrong stats key,
would pass green.","fix":"Add a fusion-hybrid test that enables the person cue, sets
min_players=2, mocks detections so window A has >=2 in-court players and window B has <2, and
asserts only A reaches provider.analyze_video while both are still persisted. Add a second case
asserting min_players is a no-op when cue_flags.person is False."}
```

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:15:34.122Z)

I'll verify this finding by reading the actual code and tests. Let me start by examining the key files.

3. user (2026-06-13T19:15:34.597Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
```

```

7
8 - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9 GPU-free, computed from the trajectory we already have); and, only when the person cue
10 is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11 (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12 fusion (P3). The person path lazily builds ONE ``PersonDetector`` ? so the default path
13 never imports torch / touches the GPU.
14 - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15 (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16 dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17 **Persisted candidates remain the full superset** regardless, so the offline
experiment
18 harness can re-score any variant.
19
20 Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21 nothing is gated (thresholds 0, person OFF), so ``FusionSegmenter`` with default config
seeds
22 and hands off identically to ``wasb_hybrid``. Registered as ``fusion_hybrid``, NOT the
default
23 segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24 experiment harness (EXPERIMENT_HARNESS.md), never silently.
25 """
26 from typing import Any, Dict, List, Optional, Tuple
27
28 from backend.pipeline.segmenters.base import segmenter_registry
29 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30 from backend.pipeline.detectors.base import DetectorRunner
31 from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33 from backend.pipeline.detectors import rally_gate
34 from backend.pipeline.detectors import fusion_features as ff
35
36
37 class FusionSegmenter(TrajectoryHybridSegmenter):
38     """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40     family = "fusion"
41     display_label = "Fusion"
42
43     def __init__(self, db: Any, config: Any):
44         super().__init__(db, config)
45         # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46         self._person: Optional[Any] = None
47
48     @property
49     def name(self) -> str:
50         return "fusion_hybrid"
51
52     @property
53     def description(self) -> str:
54         return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                 "AI provider sets semantic boundaries.")
57
58     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")

```

```

60         if substrate == "tracknet":
61             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62             return TrackNetRunner(cfg), {
63                 "weights_path": cfg.weights_path,
64                 "queue_length": cfg.queue_length,
65                 "fusion_substrate": "tracknet",
66             }
67         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                               frame_width: float, video_path: str,
72                               idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76         # over the whole trajectory by rally_gate (camera-agnostic).
77         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
78         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80         fcfg = idx_cfg.get("fusion", {})
81         if fcfg.get("cue_flags", {}).get("person", False):
82             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83         return stats
84
85     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                         frame_width: float, video_path: str,
87                         fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109     def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                           window_stats: List[Dict[str, Any]],
111                           idx_cfg: Dict[str, Any]) -> List[int]:
112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113         (0) keep every window ? identical to the substrate. Persisted candidates are

```

```

114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126                 keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

4. user (2026-06-13T19:15:35.100Z)

```

1     """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3     Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4     keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5     substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6     persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7     """
8     from types import SimpleNamespace
9     from unittest.mock import MagicMock, patch
10
11     from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14     # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15     # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16     WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17             "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18     WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19             "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22     def _points():
23         pts = []
24         for f in range(30, 120):             # window A frames: y flips 100<->900 around the net
(~500)
25             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26         for f in range(300, 360):             # window B frames: y == net level -> deadband -> no
crossings
27             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
28         return pts
29
30
31     def _runner():
32         r = MagicMock()
33         r.healthcheck.return_value = (True, "env ok")
34         r.run_predict.return_value = "out.csv"
35         r.parse_trajectory_csv.return_value = _points()
36         r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
37         return r

```

```

38
39
40     def _run(indexing=None, provider_segments=None):
41         provider = MagicMock()
42         provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
43         db = MagicMock()
44         db.add_candidate_segment.side_effect = [101, 102, 103, 104]
45         db.add_play_segment.return_value = 200
46
47         with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
48             patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)), \
49             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
50             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
51             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
52             patch("os.environ.get", return_value="dummy_key"):
53             preg.get.return_value = provider
54             seg = FusionSegmenter(db, {"indexing": indexing or {}})
55             res = seg.process_video("v1", "clip.mp4")
56             return db, provider, res
57
58
59     def _stats_calls(db):
60         """Persisted stats dicts, in call order."""
61         return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
62
63
64     def test_net_crossings_persisted_and_person_off_by_default():
65         db, _provider, _res = _run()
66         stats = _stats_calls(db)
67         assert len(stats) == 2                                # both windows persisted
68         (superset)
69         assert all("net_crossings" in s for s in stats)
70         # Window A (rally) crosses many times; window B (held) zero.
71         assert stats[0]["net_crossings"] >= 2
72         assert stats[1]["net_crossings"] == 0.0
73         # Person cue OFF by default -> NONE of the person features present.
74         assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
75         # The 4 base motion keys are still there.
76         assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
77                     for s in stats)
78
79     def test_persisted_candidates_are_superset_of_substrate_windows():
80         db, _provider, _res = _run()
81         spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
82         assert spans == [(1.0, 4.0), (10.0, 12.0)]            # exactly the substrate windows,
unfiltered
83
84
85     def test_default_hands_off_all_windows():
86         _db, provider, _res = _run()                            # min_crossings default 0 -> no
gating
87         assert provider.analyze_video.call_count == 2
88
89
90     def test_gate_a_drops_low_crossing_window_from_handoff_only():
91         db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})

```

```

92         # Persistence is still the full superset (offline substrate intact)...
93         assert len(_stats_calls(db)) == 2
94         # ...but only the high-crossing window A reaches the AI handoff.
95         assert provider.analyze_video.call_count == 1
96
97
98     def test_person_cue_adds_features_when_enabled():
99         fake_pd = MagicMock()
100         fake_pd.detections_per_frame.return_value = {
101             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
102             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
103         }
104         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
105             return_value=fake_pd):
106             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
107             stats = _stats_calls(db)
108             assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
109             assert all("net_crossings" in s for s in stats)
110             fake_pd.detections_per_frame.assert_called() # the person detector WAS used

```

5. assistant (2026-06-13T19:15:39.001Z)

Now let me verify the claims in the finding. I've read both files. Let me check the existing tests to confirm what Gate B coverage exists, and verify the key behaviors by checking the base class for how `\_select\_for\_handoff` and `players\_in\_court` work.

6. user (2026-06-13T19:15:39.639Z)

```

docs\EXPERIMENT_HARNESS.md:174:2. **Persist per-cue features** into candidate `stats` (person:
`players_in_court`, `two_sided_motion`,
docs\MULTI_SIGNAL_FUSION_PLAN.md:135:| `players_in_court` | median count of persistent in-court
tracks | ? 2 (singles) / 4 (doubles); 0?1 = dead time |
backend\eval\experiment.py:92:           whose persisted ``players_in_court`` is >= this. 0 = off
(no-op until the person
backend\eval\experiment.py:198:           col = _feature_column(vc, "players_in_court")
docs\QUALITY_ITERATIONS.md:65:player-count-only; supply ? masked). Person features
(`players_in_court`, `two_sided_motion`,
backend\pipeline\detectors\fusion_features.py:94:def players_in_court(counts: Sequence[int]) ->
float:
backend\pipeline\detectors\fusion_features.py:191:     "players_in_court", "two_sided_motion",
"mean_player_motion",
backend\pipeline\detectors\fusion_features.py:205:         "players_in_court":
players_in_court(counts),
backend\pipeline\segmenters\fusion_hybrid.py:124:             if person_on and min_players > 0
and st.get("players_in_court", 0.0) < min_players:
tests\test_experiment.py:106:         features={"players_in_court": [2.0, 1.0]})
tests\test_fusion_features.py:59:     assert ff.players_in_court(counts) == 1.0 #
median of [0,1,1,2]
tests\test_fusion_features.py:64:     assert ff.players_in_court([]) == 0.0

```

7. user (2026-06-13T19:15:41.202Z)

```

105:         db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})

```

8. assistant (2026-06-13T19:15:46.246Z)

The grep on `tests/test\_fusion\_hybrid.py` for Gate B terms (`min\_players`, `players\_in\_court`, etc.) returns only one hit: line 105, which is the `cue\_flags.person: True` in `test\_person\_cue\_adds\_features\_when\_enabled`. That test does NOT set `min\_players` and does NOT assert anything about handoff filtering based on player count ? it only checks that person features get persisted.

Let me confirm there are no other test files that might exercise Gate B, and verify

`players\_in\_court` is the correct stats key produced by the person features path.

9. user (2026-06-13T19:15:47.832Z)

```
tests\test_config.py:35:    assert f.min_players == 0                # Gate B OFF
tests\test_experiment.py:82:    min_crossings=0, min_players=0)
tests\test_experiment.py:107:    preds = predict(Variant(name="gateB", min_players=2), vc)
tests\test_fusion_features.py:60:    assert ff.in_court_ratio(counts, min_players=2) ==
pytest.approx(0.25)
tests\test_fusion_features.py:61:    assert ff.in_court_ratio(counts, min_players=1) ==
pytest.approx(0.75)
tests\test_trajectory_hybrid_equivalence.py:124:    assert seg._select_for_handoff([cw, cw, cw],
[stats, stats, stats], {}) == [0, 1, 2]
```

10. user (2026-06-13T19:15:48.485Z)

```
185         return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                frame_width: float, frame_height: float,
198                                court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                net: Optional[Net] = None,
200                                min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }
212
```

11. assistant (2026-06-13T19:16:03.349Z)

My verification is complete. Let me consolidate the findings.

****Verifying the finding's claims against code:****

1. *****`\_select\_for\_handoff` has two gates*** ? CONFIRMED.** `fusion\_hybrid.py:122` is Gate A (`min\_crossings`), `fusion\_hybrid.py:124` is Gate B (`min\_players` + `players\_in\_court`).

2. *****Gate A is covered by test_gate_a_drops_low_crossing_window_from_handoff_only*** ? CONFIRMED.** `test\_fusion\_hybrid.py:90-95` sets `min\_crossings: 2` and asserts `provider.analyze\_video.call\_count == 1`.

3. *****Gate B has no test at all*** ? CONFIRMED.** The grep across the entire `tests/` dir for Gate B terms shows:

- No test in `test_fusion_hybrid.py` sets `min_players` or asserts on `players_in_court`-driven handoff filtering. The only `cue_flags.person:True` test (line 98-109) asserts feature persistence, never handoff selection, and never sets `min_players`.
- `test_config.py:35` only asserts the default `min_players == 0` (config default, not gate behavior).
- `test_experiment.py:107` exercises a *different* code path ? `predict(Variant(...min_players=2...))` in the offline experiment harness (`backend/eval/experiment.py`), NOT `FusionSegmenter._select_for_handoff`.
- `test_trajectory_hybrid_equivalence.py:124` calls `_select_for_handoff` on the `**base class**` `TrajectoryHybridSegmenter` with empty config `{}`, which doesn't have Gate B at all (Gate B is overridden only in `FusionSegmenter`).

So Gate B in `FusionSegmenter._select_for_handoff` is genuinely never exercised by any test.

4. `***the important guard that min_players is IGNORED when person_on=False***` ? CONFIRMED. Line 124: `if person_on and min_players > 0 and ...`. The `person_on` conjunct is real and untested. A regression dropping `person_on` would change behavior but pass all tests.

5. `**Key name correctness**` ? `players_in_court` is indeed the key produced by `compute_person_features` (`fusion_features.py:205`) and read at `fusion_hybrid.py:124`. The finding's proposed test would catch a wrong-key regression.

The finding accurately quotes file:line, the logic description matches the code, and the test-gap claim holds against an exhaustive grep. This is a real, non-trivial coverage gap in a served gate that drops candidates from the paid-AI handoff. The "high" severity is slightly generous for a pure test-coverage gap (no current bug), but the finding itself explicitly frames it as untested-regression-risk, not a live bug, and the gated code is behavior-affecting (controls what reaches the paid provider). I'll confirm it as real, with a note that severity is more medium than high.

12. user (2026-06-13T19:16:15.634Z)

Structured output provided successfully

13. assistant (2026-06-13T19:16:24.988Z)

I've completed the adversarial verification. The finding is `**real**` (confirmed against the code), with severity adjusted to `**medium**`.

Summary of what I verified:

- `backend/pipeline/segmenters/fusion_hybrid.py:124` ? Gate B is `if person_on and min_players > 0 and st.get("players_in_court", 0.0) < min_players: continue`. This exists exactly as the finding quotes.
- Gate A (line 122) is tested by `test_fusion_hybrid.py:90-95`; Gate B has `**no test**`. An exhaustive grep of `tests/` confirms the only person-cue test (`test_person_cue_adds_features_when_enabled`, lines 98-109) asserts feature *persistence* only ? it never sets `min_players` and never asserts handoff selection.
- The other `min_players` hits are unrelated: `test_config.py:35` (config default), `test_experiment.py:107` (the offline `backend/eval/experiment.py` predict path ? a different code path), and `test_trajectory_hybrid_equivalence.py:124` (base class, where Gate B doesn't exist).
- The two specific regression risks hold: the `person_on` guard and the `players_in_court` key read (matches `fusion_features.py:205`) are both unasserted, so a regression in either would pass green.

Severity downgraded from high to medium: it's a pure coverage gap (the gate logic is correct as written, no live bug) on an opt-in, default-OFF segmenter with `min_players` defaulting to 0. It's still behavior-affecting because Gate B controls which candidates reach the paid AI provider.